

5

Extending Your Agent with RAG to Prevent Hallucinations

In earlier chapters, we saw what an LLM is, and in the previous chapter, we saw how it can control different tools to succeed at completing a task. However, some of the limitations of LLMs prevent their deployment in sensitive fields such as medicine. For example, LLMs crystallize their knowledge at the time of training, and rapidly developing fields such as medical sciences cause this knowledge to be outdated in a short time. Another problem that has emerged with the use of LLMs is that they can often hallucinate (produce answers that contain factual or conceptual errors). To overcome these limitations, a new paradigm has emerged: **retrieval-augmented generation (RAG)**. RAG, as we will see in this chapter, allows for the LLM to refer to memory that is external to the model; thus, it allows knowledge to be found and kept updated. Similarly, providing contextual guidance to the model's response allows for the reduction of hallucinations. Therefore, RAG is widely used today and is considered a promising system.

In this chapter, we will discuss how this system has evolved, starting with how a transformer can be used to find information. We will discuss in detail the various components of the system (embedding, vector database, and generation).

In this chapter, we'll be covering the following topics:

- Exploring naïve RAG
- Retrieval, optimization, and augmentation
- Evaluating the output
- Comparison between RAG and fine-tuning
- Using RAG to build a movie recommendation agent

Technical requirements

Most of this code can be run on a CPU, but it is preferable to be run on a GPU. The code is written in PyTorch and uses standard libraries for the most part (PyTorch, Hugging Face Transformers, LangChain, SentencePiece, Datasets, and scikit-learn). The code can be found on GitHub: <https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr5>.

Exploring naïve RAG

Information retrieval is the name of the scientific field that deals with finding information in media (often textual but also multimodal). For example, the user may be interested in finding whole documents or chunks

in documents; this task is key to question answering, where a model has to find the steps needed to answer a user's questions. At the heart of the system is a search engine. In the case of RAG, the search engine is a transformer (or at least a language model), and in this chapter, we will focus on that. We will discuss a system in which we have a **collection** of documents (textual, but could also be web pages, images, videos, or even code or short text passages) that have corresponding indexes in the database. These documents can be associated with metadata (attributes describing author, size, topic, and keywords). By convention, a **term** is defined as a word present in the text but also a passage that can answer the search. A user produces a **query** that can be expressed as terms. The purpose of the retrieval system is to best match the query with the relevant documents in the collection. These are then returned in order of relevance.

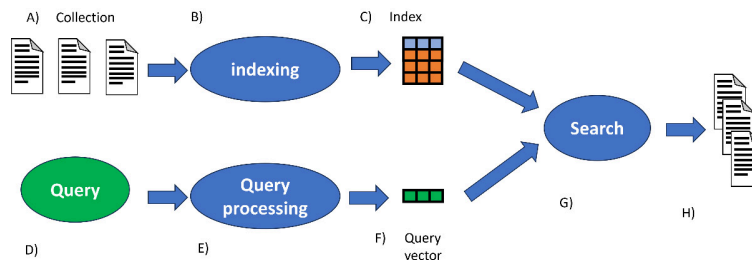


Figure 5.1 – Workflow diagram showing how a query is processed by a search

Let's break down what we can see in *Figure 5.1*. A collection of documents (A) is indexed (B) and is entered in an orderly manner into a database (C). Each document is assigned metadata and indexes. A user query (D) is processed (E) to obtain a vector representation (F). The resulting vector is used during the search to find the documents that are most relevant (G). The system returns the documents in order of relevance (H)

As we can observe, the system uses a search in a vector space. In the simplest form, this can be bag-of-words or the TF-IDF we saw in the first chapter. For example, we can take a set of documents and calculate the TF-IDF. Once we've done that, we can calculate a score (usually cosine similarity) between each of the documents and conduct rank based on the score. For a document d and a query q in vector form, we use the following formula:

$$\cos(q, d) = \frac{q \cdot d}{|q| |d|}$$

We can see an example of this process here:

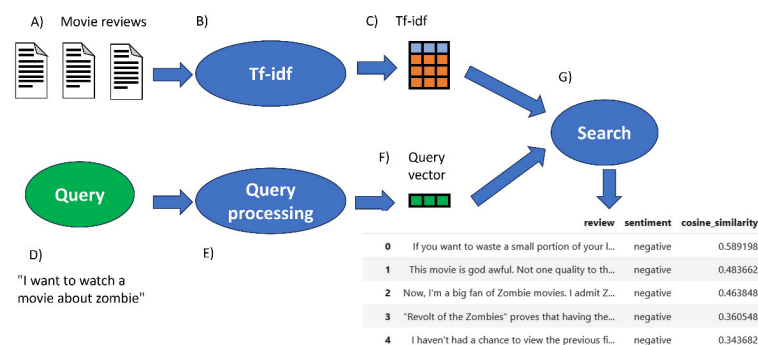


Figure 5.2 – Example of retrieving the most relevant documents with TF-IDF

This type of research also requires data storage facilities that are suitable. For example, for TF-IDF (or derivative algorithms), an inverted index is used as the data structure. The inverted index is a data structure designed specifically to make it efficient to search for terms in a set of documents. It is a structure composed of a dictionary and postings. The dictionary indicates the frequency of terms and the posting in which document they are found. In this way, given a set of terms in the query, we can efficiently find the documents that contain them and calculate the similarity.

```

Hello {1}      -> 2[1]
Morning {2}    -> 2[2] -> 5[1]
Today {3}      -> 3[1] -> 4[2] -> 6[1]
Tomorrow {2}   -> 2[3] -> 3[1]

```

Figure 5.3 – Example of an inverted index

BM25 is a variant of TF-IDF where two parameters are added: b , which controls the importance of document length normalization, and k , which controls the relationship between **term frequency (TF)** and **inverse document frequency (IDF)**.

$$BM25score = \sum_{t,q} \overbrace{\log\left(\frac{N}{df_t}\right)}^{IDF} \overbrace{\frac{tf_{t,d}}{k(1-b+b(\frac{|d|}{|d_{avg}|})) + tf_{t,d}}}^{TF}$$

The preceding equation is a variation of TF-IDF for a document d and a query q (d_{avg} represents the average length of a document).

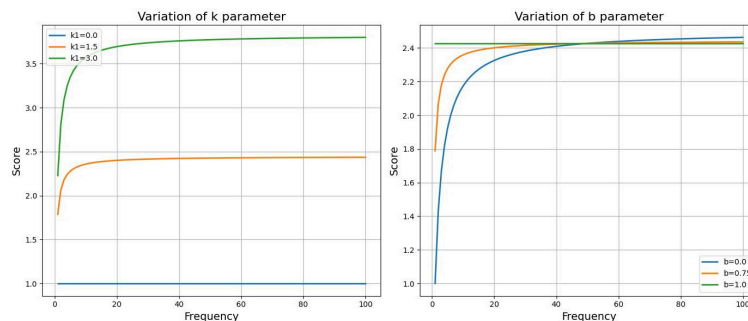


Figure 5.4 – Effect of k and b parameters on the BM25 score

We can see some interesting points:

- By selecting k equal to zero, no TF is used in the score. The TF component becomes irrelevant; the score does not consider how often a term appears in a document, only whether it appears at all. Higher k values give greater weight to TF. k is used to adjust TF saturation – in other words, how much a single query term impacts the score of a single document. $b=1$ means normalize for document length, while 0 means eliminate normalization.
- The system is sensitive to both TF and document length without adding too many parameters. The usually recommended values are $b=0.75$ and k between 1.2 and 2. BM25 is much more flexible than TF-IDF and can be adapted to different scenarios.
- It is not much more complex than TF-IDF and is therefore scalable to large datasets, and more robust to sparse matrices.

It is not always easy to find the optimal parameters for a precise dataset. The model is sensitive to the choice of hyperparameters. BM25 has a limited understanding of semantics since it is based on term frequency, not capturing the meaning of a document. Also, many terms are polysemous (with multiple meanings) and BM25 does not capture the context of a term. Another serious problem is a vocabulary mismatch problem – that is, when there is no complete overlap between terms in the query and documents.

The solution to these problems is to use dense vectors that include contextual information. This is done by using a transformer and extracting the representation for a document. More formally, given a sequence of tokens, we use the representation z , obtained from the final layer. This allows us to obtain a high-dimensional representation that we can use to disambiguate the meaning of a word. This is called the **z-score**.

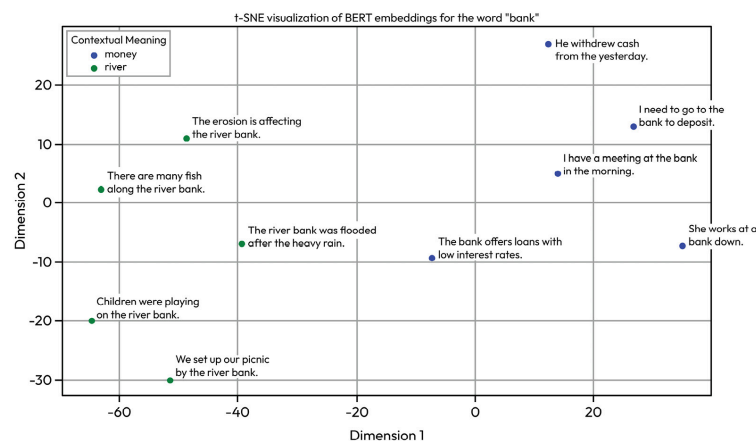


Figure 5.5 – Contextual embeddings for the word “bank”

Figure 5.5 shows a **t-distributed stochastic neighbor embedding (t-SNE) visualization** of the contextual embedding for the word “bank” in different contexts (both money and river-related meanings). The t-SNE is conducted on the BERT embedding of the word for each sentence.

There are several ways to retrieve this representation from the model. For convenience, the last layer is used, but it is generally proposed to conduct an average pool of the representation of multiple layers (each block learns a different text representation due to self-attention). As we saw in [Chapter 1](#), these vectors have geometric properties and can be used for operations (clustering, similarity computation, and so on).

Generally, some transformation is conducted to optimize the use of embedding. For example, a normalization of vectors (z-score or other methods) is conducted. In fact, the vectors of many words are similar due to anisotropy. In fact, taking random words, the cosine similarity is higher than it should be. This is due to rogue dimensions, a small number of dimensions (1–5) that dominate contextual embedding because they have high magnitude and disproportionately high variance. This causes similarity to be calculated on reduced embedding space. These rogue dimensions are highly correlated with absolute position and punctuation and are therefore uninformative. Transformations such as z-score can reduce the problem.

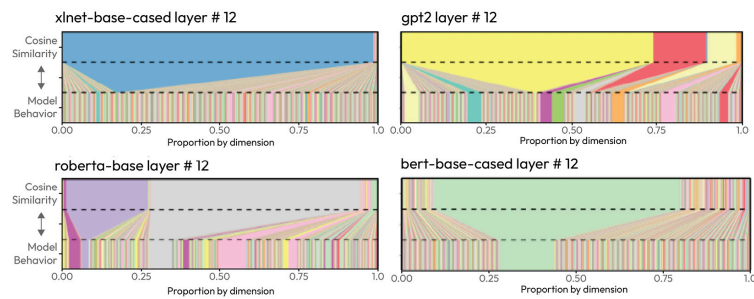


Figure 5.6 – Relative contribution of each dimension to cosine similarity

(<https://aclanthology.org/2021.emnlp-main.372.pdf>)

In addition, retrieving the embedding for each word in the embedding is unnecessarily laborious. For bidirectional encoders, we can use two main strategies: use a single encoder or a bi-encoder. In the first case, we provide the model with both query and document, thus allowing bidirectional self-attention to attend all tokens. The representation will be representative of both the query and the document. The format used is [CLS]-query-[SEP]-document. The representation for the [CLS] token is then fed to a linear layer to produce the similarity score (this layer is fine-tuned). Normally, this process is done not for the whole document but for a series of chunks (non-overlapping fragments of the document), because documents are usually longer than the context length (for BERT, this is 512 tokens, so the sum of query and document must be no more than 512 tokens).

This system is expensive because it requires that we have to pass a query along with the entire corpus of documents. To reduce the cost, a more efficient architecture known as a bi-encoder was implemented. One encoder is used to extract the representation for the query, [CLS]q, and another to extract the representation for each document (or chunk), [CLS]d. Basically, taking a corpus, we compute the embedding for each document in the corpus and store this representation in a database. After that, we compute the cosine similarity between the representation for the query and all the vectors in the database. This system is much faster but less accurate because part of the interactions there are between the terms in the query and in the documents.

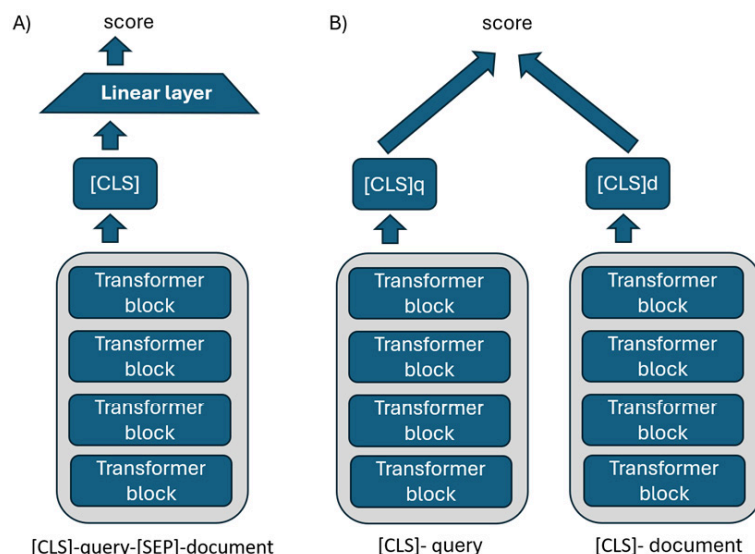


Figure 5.7 – Two different approaches for contextual embedding

Let's examine *Figure 5.7* in more detail. **A** is a unique model for generating the contextual vector. A linear layer is fine-tuned to generate a score

of similarity between query and vector. The linear layer has as input the **[CLS]** representation. **B** shows two models, which are used to generate two different and separate representations. The **[CLS]** representation is generated for the query and all the vectors. We calculate cosine similarity using both representations.

Then, we can conduct embedding of the whole corpus and index the documents in a database and then, when a query comes, calculate the similarity.

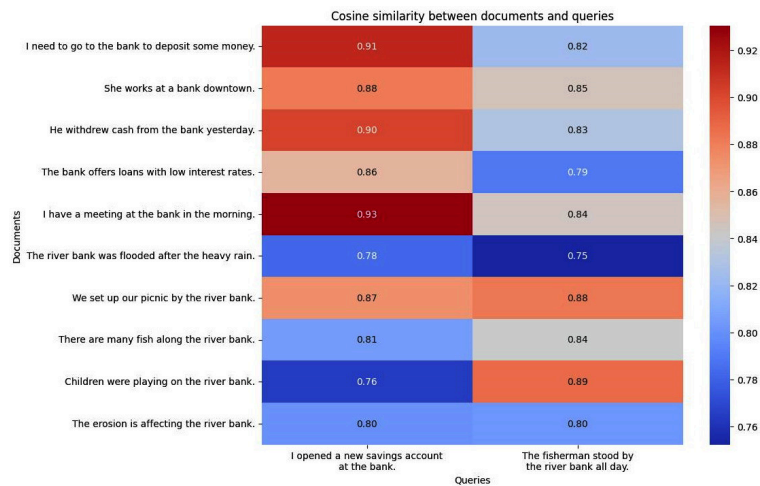


Figure 5.8 – Cosine similarities between a set of documents and two queries where the meaning of the word “bank” is different

As we mentioned earlier, generative models can produce hallucinations. Given a query, LLMs can generate output that contains erroneous information. This stems from the fact that LLMs are good at explaining concepts but have problems retaining specific information. During training, knowledge of a concept is reinforced by the repetition of similar pieces of information. This works well for concepts but less so for specific pieces of information such as dates, numerical values, and rare pieces of information. In addition, datasets contain both correct and incorrect information, often conflicting. When a model generates a response, it samples from a distribution and must choose from the information it has learned, thus leading to hallucinations.

In addition, incorrect architecture, overfitting, or misalignment during training can also promote hallucinations. Fine-tuning the model or over-optimization for some tasks can be an additional cause. For example, optimizing the model to write long text outputs promotes the model to become verbose and generate hallucinations. Similarly, raising the temperature increases the stochasticity of sampling, leading to sample tokens that are less likely and thus hallucinate more. Incorrect prompting can also promote this behavior.

Hallucinations are most evident when using a model in a specific domain (healthcare, finance, and so on). The model lacks the context to best understand the query. This is because the model has been trained on a huge number of tokens, and they have not been restricted to specialized topics. The loss is calculated on the set of texts and thus more on general knowledge than on particular information. Therefore, the model favors a generalist function but performs less well when applied to a particular domain. This is a common factor, irrespective of the number of model parameters.

Several possible solutions have been tested to reduce or prevent hallucinations. One approach is to provide context as part of the LLM prompt (when it is possible to add all this context to the prompt). However, this means that the user has to find the relevant context again. When you have many different documents, this becomes a complex and laborious task. Alternatively, fine-tuning, in which the model is trained further on specific documents, has been proposed. This has a computational cost, though, and should be conducted repeatedly if new documents arrive.

In 2020, Meta proposed an alternative approach: RAG for LLMs. This approach assumes augmenting the generation of an LLM by finding the context in an external source (such as a database). This database can be domain-specific and continuously updated. In other words, we find the documents needed to answer the query and take advantage of the fact that an LLM has powerful abilities for in-context learning.

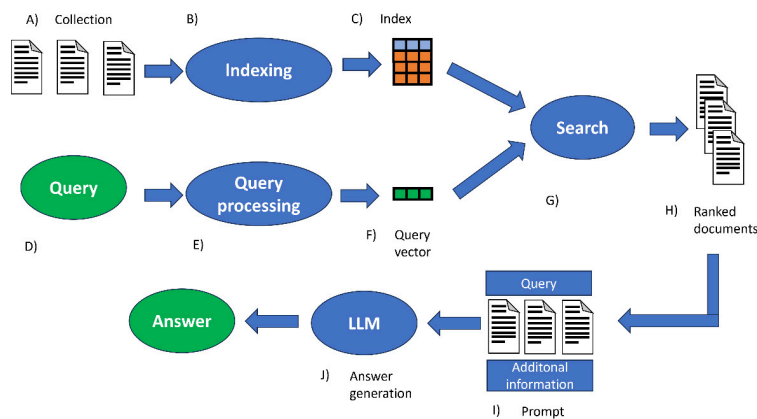


Figure 5.9 – Diagram showing the process of RAG

In *Figure 5.9*, the ranked documents are incorporated in the prompt (query, retrieved documents, and additional information), which is presented to the LLM. The LLM uses the additional context to respond to the query of the user.

We define the knowledge from the LLM as parametric memory and that obtained from the RAG as external or nonparametric. More formally, RAG is a system that, in its most basic form, consists of three parts:

- **Indexing:** Indexing deals with the entire process from raw data to storage in a vector database. It begins with ingesting data in various formats (PDF, HTML, Markdown, or XML) that must be converted to text. The text is processed according to the embedding model chosen (it is divided into chunks that must be smaller in size than the context length of the model). The chunks are then embedded (transformed into a vector representation), assigned an identifier, and stored in a vector database.
- **Retrieval:** When a query arrives, the most relevant chunks must be found. The same encoder used for document embedding is used to obtain a vector for the query. The similarity score between the query vector and the vectors stored in the database is then calculated. Top *K* chunks are selected based on the similarity score.
- **Generation:** The chunks found together with the query are incorporated into a consistent prompt for LLM used for generation. Different LLMs may require different elements in the prompt to work best; similarly, we can have prompts that are tailored for specific tasks. In addition,

tion, we can also add elements from the previous conversation (history).

For an autoregressive model, we can modify the equation seen in [Chapter 2](#), where we defined that an LLM computes the probability of a sequence of tokens given the previous tokens:

$$p(x_1, x_2, \dots, x_n) = \prod_{i=1}^n p(x_i | x_{<i})$$

For a question-answering task, given a question (or query) q , we can rewrite the equation as follows:

$$p(x_1, x_2, \dots, x_n) = \prod_{i=1}^n p(q; x_{<i})$$

In RAG, we have additional elements: the prompt Pr , the context retrieved R , and the question q , which are all concatenated:

$$p(x_1, x_2, \dots, x_n) = \prod_{i=1}^n p(x_i | Pr; R; q; x_{<i})$$

This process is shown in [Figure 5.10](#):

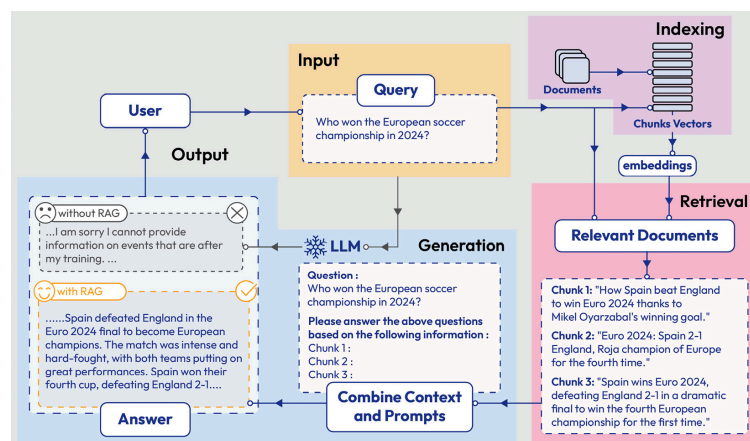


Figure 5.10 – Representative instance of the RAG process and its steps

(<https://arxiv.org/pdf/2312.10997>)

This is the general architecture, but there are also more complex variations (which we will discuss in detail in the next chapter). For completeness, an alternative to this architecture is **span extraction**. In this case, instead of finding the most appropriate chunks, we have a language model (usually also derived from BERT) that is used to find passages in the text that answer a query (**span labeling**). For example, if our corpus is Wikipedia and our query is “*Who is the president of France?*”, the extractor will label the passage on the page that answers the question (in RAG, we retrieve the text chunks that are relevant instead). RAG (or **span extractor**) has shown interesting abilities in reducing hallucinations and improving the abilities of LLMs in open-domain question answering (also called open-book QA).

In the next section, we will go on to discuss these steps in more detail and what choices we need to make in order to optimize the system.

Retrieval, optimization, and augmentation

In the previous section, we discussed the high-level RAG paradigm. In this section, we are going to look at the components in detail and analyze the possible choices a practitioner can make when they want to implement a RAG system.

Chunking strategies

We have stated that text is divided into chunks before being embedded in the database. Dividing into chunks has a very important impact on what information is included in the vector and then found during the search. Chunks that are too small lose the context of the data, while chunks that are too large are non-specific (and present irrelevant information that also impacts response generation). This then impacts the retrieval of query-specific information. The larger the chunking size, the larger the amount of tokens that will be introduced into the prompt and thus an increase in the inference cost (but the computational cost of the database also increases with the number of chunks per document). Excessive context can also lead to hallucinations and detract from LLM performance. In addition, the chunk size must not exceed the context length of the embedder, or we will lose information (this is known as truncation). In other words, chunk size is an important factor that affects both the quality of retrieval and generation.

The simplest strategies are those based on a fixed length of chunking. Character chunking divides the document into chunks based on a predetermined number of characters or tokens (common choices are 100 or 256 tokens or 500 characters). The size should be chosen according to the type of document. This is the cheapest and easiest system to implement. One variation is a random chunk size where the size of the chunks is variable. This variant can be used when the collection is non-homogenous and potentially captures more semantic context. Separation into chunks can be with or without overlap. Chunking without overlap (*Figure 5.11*) works well if there are clear boundaries between chunks (such as if the context changes drastically between adjacent chunks). This is rarely the case, though, and the lack of overlap destroys context. One can then use a sliding window that maintains an overlap between chunks. This system maintains contextual information at the chunk boundaries, allowing better semantic content and increasing the chance that relevant information will be found if it spans across multiple chunks. This strategy is more expensive, though, because we need to divide it into more chunks, so we will have a database with many more entries. Also, some of the information is redundant, so the overlap should be no more than a small percentage of the entire chunk size.

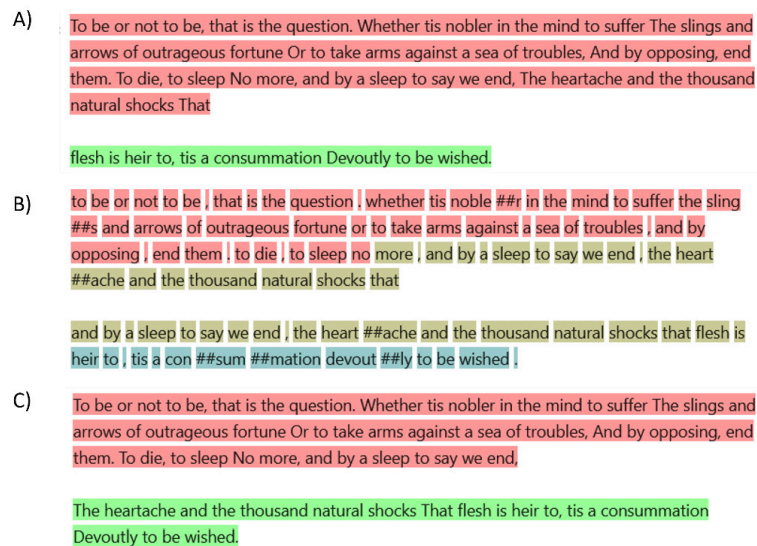


Figure 5.11 – Effect of different chunking strategies used on an extract from Hamlet

In *Figure 5.11*, we can see the following:

- **A)**: Simple chunking based on the number of tokens and without overlap
- **B)**: Simple chunking based on the number of tokens and with overlap
- **C)**: Simple chunking based on the number of tokens and the presence of the new line in the text (character-based)

Context-aware chunking is a strategy in which we divide text into chunks using a **regular expression (regex)**. For example, we can divide based on periods, commas, or paragraph breaks. Variants of this strategy are based on the type of text we are splitting (for example, HTML tags, Markdown information, XML, domain-specific signs, and so on). This system is not without its drawbacks; it can sometimes be difficult to determine boundaries (for example, for compound sentences, dirty text, and so on). You can therefore have chunks that are of varying sizes. A more sophisticated variant is called **recursive chunking**, in which the chunk is split similarly to context-aware chunks. After that, the chunks are joined up to a predetermined number of tokens (for example, the maximum context length of the embedder). This approach tries to keep all information that is contextually related in the same chunk and maintain semantic consistency (for example, if possible, all chunks belonging to a paragraph are merged). Alternatively, the text is iteratively split until the chunks reach the desired size. **Hierarchical clustering** is a similar method that seeks to respect the structure of the text. By examining relationships in the text, it tries to divide it into segments that respect its hierarchy (sections, subsections, paragraphs, and sentences). This system is useful for documents that have a complex and known structure (business reports, scientific articles, and websites). This method also makes it possible to inspect the structure obtained and to understand the relationship between the various chunks. This system works poorly when dealing with documents that are poorly formatted.

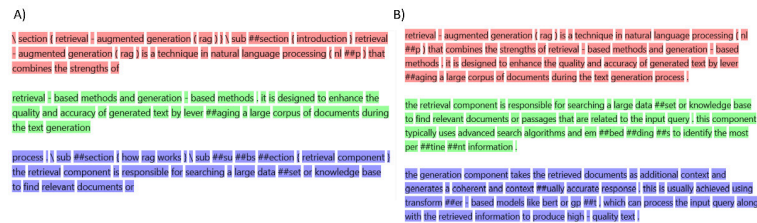


Figure 5.12 – Demonstration of hierarchical chunking

Figure 5.12 shows the same document in Markdown (A) or LaTeX (B).

Using a specific chunker, we can split respecting the language structure. LangChain uses hierarchical clustering to achieve that.

Another family of methods is **semantic chunking**. The purpose of these techniques is to take into account the context and meaning of words. These methods try to group chunks that would otherwise be distant in the text (presence of digression or other elements). **K-means chunking** is an approach in which we conduct an embedding of the various sentences, then use *k*-means clustering to group sentences that are similar into various clusters. This approach requires setting the optimal number of clusters (hyperparameters) to choose and can lead to loss of sentence order (with potential risk to chronological order or contextual relationships). Instead of considering division on sentences, **propositions-based chunking** divides on contextual understanding. So-called “propositions” are identified as atomic expressions that contain factoids (sentences that are self-contained and describe a piece of knowledge, such as “*The capital of France is Paris*”). These propositions are then evaluated by an LLM that groups them according to semantic coherence. This approach can give optimal results but is computationally expensive and depends on the choice of LLM used. **Statistical merging**, on the other hand, evaluates similarities and differences in the embedding of sentences to decide whether to merge (or split) them. For example, after embedding, the difference in statistical properties (standard deviation, percentile, or interquartile difference) is evaluated, and if it exceeds a predefined threshold, the sentences are separated. This system creates chunks of different sizes and has a higher computational cost but can give better results when contextual boundaries between sentences are unclear.

Finally, a multimodal chunk may be needed. For example, a PDF file may contain both text and images. In this case, it will be necessary for our chunking pipeline to be able to extract both images and text.

There is no universal best chunker – the best chunker is the one most suited to our specific case. We can establish guidelines though, as follows:

- **Align chunking with document structure:** Text structure heavily influences the chunk size and chunk strategy. In cases where we have documents of the same type (HTML, LaTeX, Markdown, and so on), a specific chunk might be the best choice. If they are a heterogeneous collection, we could create a pipeline that conducts chunking according to the file types.
- **Optimize for performance and resources:** If we have space and computational cost limitations, a simple fixed-size chunker might be an optimal choice. Semantic chunking is slightly less performant but better respects information integrity and improves the relevance and accuracy of found chunks. It requires knowledge of the text, though, and might not be the optimal choice for a system that is used by gener-

al users. Contextual chunking may have better performance but has a high computational cost.

- **Respect model context limitations:** Chunk size should respect the dimension of the context length. We must take into account both the size of the embedding model and the LLM that we will then use to generate it.
- **Match chunking strategy to user query patterns:** Consider the type of question we expect our potential users to ask the system (the RAG). For example, if the user is going to ask questions that require the model to find multiple facts, it is better to have a strategy with small chunks but containing a direct answer. Or if the system is more discursive, it would be better to have chunks that give more context.

In conclusion, a developer has to inspect the text and the output delivered when testing different chunking strategies for the RAG system. In any case, each strategy should then be evaluated (in later sections, we will discuss how to evaluate them).

Embedding strategies

As we saw earlier, an embedding is a dense vector representation of a text (representation lying in a multidimensional space). We exploit these vectors to find the appropriate context for our query. We can have encoders that produce scattered vectors (such as TF-IDF or BM25) or encoders that generate dense encoders. As mentioned earlier, dense encoders are transformers that produce vectors. The advantage is that these models are trainable and thus can be adapted to the similarity task between queries and chunks. BERT-based backbone is one of the most widely used; the approach is to create two parallel BERT encoders (two streams: one for the query and the other for the chunk) called the bi-encoder approach. In the first RAG approaches, these weights are identical (frozen), and we only have one layer that is trained to generate the embedding vector. Having the same weights allows us to be able to pass the query first and then the documents and then calculate similarity. Later models, on the other hand, conduct fine-tuning of the weights to improve the model's ability to generate better embedding vectors.

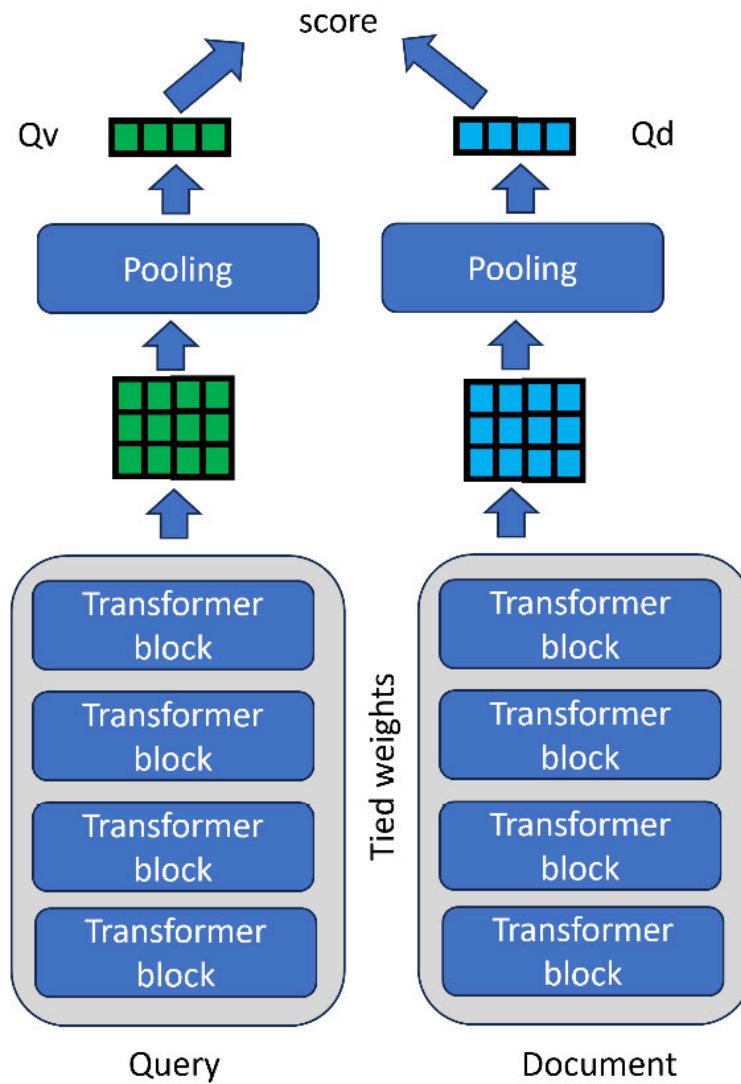


Figure 5.13 – Bi-encoder for generating embedding vectors

The bi-encoder shown in *Figure 5.13* generates a query vector and a document vector. On these two vectors, we can calculate the similarity.

Alternatively, a model can be trained from scratch for this task. Typically, it is better to take an LLM that has been trained unsupervised and then adapt it for embedding and retrieval. Normally, this model is adapted using contrastive learning. As we saw in [Chapter 3](#), contrastive learning is a technique used to learn semantic representations in the form of embedding. In [Chapter 3](#), we used CLIP, which was trained using images and captions. In this case, we want to train a model that generates embeddings that allow us to find the documents that are most akin to our query. One of the most used datasets is the Multi-Genre Natural Language Inference (MultiNLI) corpus, which contains 433,000 sentence pairs annotated with textual entailment information. Given a hypothesis, a second sentence represents an entailment, a contradiction, or neither (neutral).

In contrastive learning, we need positive and negative examples. Having taken a sentence, we want the embedding of our sentence to be as close to a positive example as possible and as dissimilar from a negative example. In this case, we can derive positive and negative examples for one of our sentences from MultiNLI. In fact, an entailment sentence represents a positive example while a contradiction is a negative example.

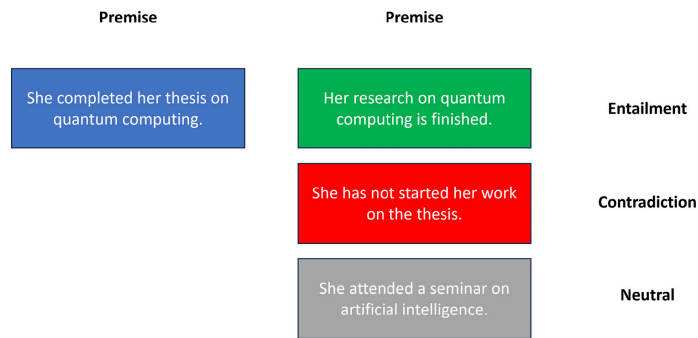


Figure 5.14 – Example of sentences that are in entailment or contradiction and can be used for training an encoder

Once we have the dataset, these models are trained with a loss function that is suitable for the task:

- **Cosine similarity loss:** This is one of the loss functions that is most commonly used because it measures the semantic closeness of two sentences. The model is optimized to give a similarity as close as possible to **1** for sentences that are similar (original sentence and positive example) and **0** for sentences that are dissimilar (original sentence and negative example). As a loss, we calculate the similarity between the two sentences, and then it is compared with the predicted label.
- **Multiple negatives ranking loss:** This is another popular alternative (also called **InfoNCE**). Only positive examples are used for this type of loss. In this case, we have the original sentence and the corresponding positive example (the entailment sentence). For negative examples, we take our original sentence and a sentence that is in entailment for another sentence. After that, we calculate embedding and similarity. The idea is to maximize the similarity between a sentence and one that is related (its positive example) while minimizing the similarity with examples that are unrelated (our negative examples). In this way, this task becomes a classification task, and we can use cross-entropy. However, the negative examples are completely unrelated, and thus the task can be too easy for the model (instead, it is better to add negative sentences that are related but not the right answer).

The choice of embedder is a critical decision that will strongly impact the performance of our system. A poor embedder will lead to poor retrieval and context not relevant to the query, which, paradoxically, could increase the risk of hallucinations. The encoder choice impacts the following:

- **Cost:** An embedder is a transformer. The bigger it is, the higher the computational cost. A closed-source encoder, on the other hand, has a cost relative to the API, so the more it is used, the greater the cost. In addition, there are computational costs associated with embedding documents and also with each query.
- **Storage cost:** The larger the size of the embedded vectors, the higher the storage cost of our vectors.
- **Latency:** Larger models have higher latency.
- **Performance:** The cost of some choices is justified if our major concern is performance. Often, larger models have better performance.
- **Domain requirements:** There are now specialized encoders for some domains (finance, medicine, science, programming, and so on) and some are multilingual (most support only English but others support

up to a hundred languages). Some domains have different text granularity and require models that are specialized for long text.

Deciding which encoder model to use is not easy and depends on various factors. A good way to start is the **MTEB leaderboard** on Hugging Face (<https://huggingface.co/spaces/mteb/leaderboard>), which is an up-to-date list of encoding models and their performance on different benchmarks and tasks. Often, though, these results are self-reported and are obtained on standard benchmarks (some of this benchmark data may be leaked in the training data and thus overestimate the model's capabilities). Thus, we should not choose just one model but test several on one's dataset. The leaderboard, however, provides some important information that allows us to guide our choice:

- **Retrieval average:** Calculates **normalized discounted cumulative gain (NDCG)** across several datasets (an evaluation metric used for ranking retrieval systems)
- **Model size:** This gives us an insight into the computational cost and resources we need to use it
- **Max tokens:** The number of tokens that can be used in the context length
- **Embedding dimensions:** Considers the size of the vectors after embedding

Overall MTEB English leaderboard

Metric: Various, refer to task tabs
Languages: English

Rank	Model	Model Size (Million Parameters)	Memory Usage (GB, fp32)	Embedding Dimensions	Max Tokens	Average (56 datasets)	Classification Average (12 datasets)	Clustering Average (11 datasets)	PairClassification Average (3 datasets)
1	SFR-Embedding-2_B	7111	26.49	4096	32768	78.31	89.05	56.17	88.07
2	gte-Owen2-7B-Instruct	7613	28.36	3584	131072	78.24	86.58	56.92	85.79
3	neutral-embedding-v1					69.94	87.91	54.32	87.68
4	NV-Embed-v1	7851	29.25	4096	32768	69.32	87.35	52.8	86.91
5	voyage-large-2-Instruct			1024	16000	68.28	81.49	53.35	89.24
6	Lin-Embed-Mistral	7111	26.49	4096	32768	68.17	88.2	51.42	88.35
7	SFR-Embedding-Mistral	7111	26.49	4096	32768	67.56	78.33	51.67	88.54
8	gte-Owen1.5-7B-Instruct	7099	26.45	4096	32768	67.34	79.6	55.83	87.38
9	gte-Owen2-1.5B-Instruct	1776	6.62	4096	131072	67.16	82.47	48.75	87.51
10	gte-Owen2-1.5B-Instruct-0B-B					67.16	82.47	48.75	87.51

Figure 5.15 – MTEB leaderboard dedicated to embedding models and their performance

It should also be noted that the leaderboard measures on generic domains. This means it measures general performance, which could result in poor performance in our domain or task of interest.

Once we have selected an encoder, we can reduce this cost without affecting the performance. Regarding cost and scalability, for each dimension of the vector embedding, we need 4 bytes of memory if they are in a float format. This can lead to exorbitant costs in storage. In [Chapter 3](#), we discussed quantization – this can also be applied to embedding models.

Binary quantization (which reduces models to 1 bit per dimension) can lead to a reduction in memory and storage of up to 32 times. The simplest binary quantization is to use a zero threshold:

$$f(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ 1 & \text{if } x > 0 \end{cases}$$

We can then use the **Hamming distance** to find the value for the weights more efficiently (Hamming distance is another evaluation metric most suitable for these binary vectors. You can read more about it using the link in the *Further reading* section at the end of the chapter). Binary en-

coding allows unprecedented speed and memory reduction but generally means a loss of performance. More sophisticated versions of this approach can maintain up to 96% similarity (approaches on re-scoring found chunks). This quantization is considered extreme; often, a good compromise is to go from the **float32** format to **int8** (a format in which we represent values using 256 distinct levels). As we described earlier, this is done by recalibrating the vectors during the transformation.

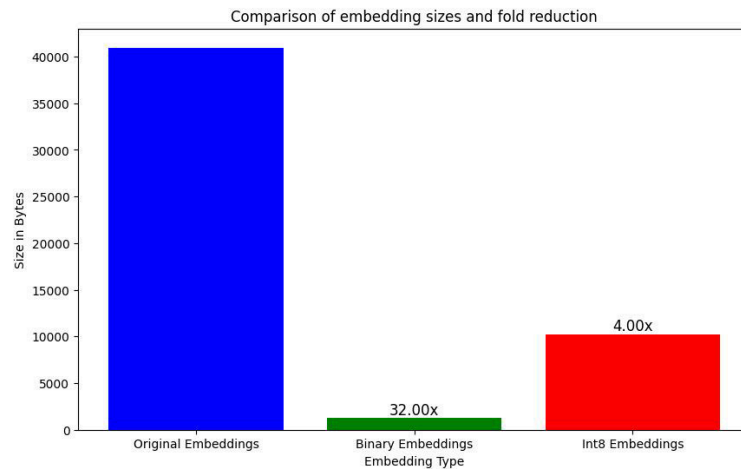


Figure 5.16 – Graph showing memory deduction after quantization

An alternative technique is Matryoshka Representation Learning. Deep learning models tend to spread the information over the entire vector; this technique attempts to compress the information over several representations with fewer dimensions instead. In other words, it progressively reduces the divisions of vector embeddings without losing too much performance. In a Matryoshka embedding, smaller embeddings are obtained that can be used as larger embeddings. This is because the system tries to force storage of the most important information in early dimensions and less important information in later dimensions (in this way, we can truncate the vector while maintaining performance in downstream tasks). To train an encoder, we produce embeddings for a batch of text and then compute the loss. For Matryoshka embedding models, the loss also takes into account the quality of the embedding at different dimensionalities. These values are summed in the final loss. Thus, the model tries to optimize the model weights in a way that the most important information (for the embedding vectors) is located in the first dimensions.

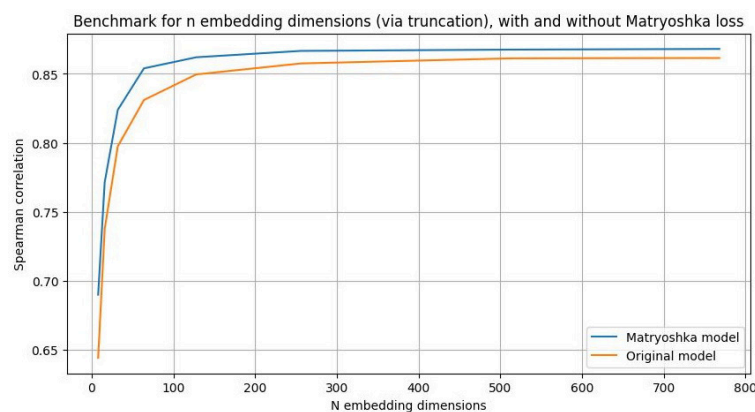


Figure 5.17 – Benchmark of Matryoshka versus original embedding quality over number dimensions

Once we have our vectors, we need to store them. In the next section, we will discuss where to store them.

Embedding databases

A vector database is a specialized database for the storage of high-dimensional vectors. This database is therefore optimized for handling unstructured and semi-structured data such as vectors. The function of this database is to allow efficient storing, indexing, and searching. The vector database we choose also has a big impact on RAG performance. Today, there are dozens of possible vector databases, so choosing the best solution can be a daunting task. Fortunately, there are sites that conduct a comparison of possible systems.

Vector DB Comparison
by Superlinked | Last Updated: 1 days ago

Search

Vendor	About				Search									
	OS	License	Dev Lang	VDS Launch	Filters	Hybrid Search	Facets	Geo Search	Multi-Vector	Sparse	BM25	Full-Text		
09 Activeloop...	🟢	🟢	Python	2023	🟢	🟢	-	🟢	🟢	🟢	🟢	🟢	🟢	🟢
09 Aeroglide	🔴	Proprietary	Java	2024	🔴	🔴	🔴	🔴	🟢	🟢	🔴	🔴	🔴	🔴
09 Anari AI	🔴	Proprietary	C	2023	-	🔴	-	🔴	-	-	🔴	-	-	-
09 Apache Ca...	🟢	Apache 2.0	Java	2023	🟢	🟢	🟢	-	-	-	🔴	-	🔴	🔴
09 Apache Solr	🟢	Apache 2.0	Java	2022	🟢	🟢	🟢	🟢	🟢	-	🟢	🟢	🟢	🟢
09 ApertureDB	-	🟢	-	-	-	-	-	-	-	-	-	-	-	-
09 Azure AI S...	🔴	Proprietary	C++	2023	🟢	🟢	🟢	🟢	🟢	🔴	🟢	🟢	🟢	🟢
09 Chroma	🟢	Apache 2.0	Rust	2022	🟢	🔴	-	-	🔴	🔴	🔴	🔴	🟢	🟢
09 Clickhouse	🟢	Apache 2.0	C++	2022	🟢	🟢	🟢	🟢	🟢	🟢	🔴	🔴	🔴	🔴
09 Couchbase	🟢	🟢	🟢	2024	🟢	🟢	🟢	🟢	🟢	🟢	🟢	🟢	🟢	🟢
09 CrateDB	🟢	Apache 2.0	Java	2023	🟢	🟢	-	🟢	🟢	🟢	-	🟢	🟢	🟢
09 DataStax A...	🔴	Proprietary	Java	2023	🟢	🟢	🟢	🟢	🔴	🔴	🔴	🔴	🟢	🟢
09 Elasticsearch	🔴	Static License	Java	2021	🟢	🟢	🟢	🟢	🟢	🟢	🟢	🟢	🟢	🟢
09 Epsilon	🟢	GPL-3.0	C++	2023	🟢	🟢	-	-	🟢	🟢	🟢	🟢	-	-
09 GCP Verta...	🔴	🟢	C	2021	🟢	🔴	-	-	🔴	🔴	🔴	🔴	🔴	🔴
09 HypeSpace	🔴	Proprietary	Python	2023	🟢	🟢	🟢	🟢	🟢	🟢	-	🔴	🟢	🟢

Figure 5.18 – Vector DB leaderboard, a practical source to address the vector database choice (<https://superlinked.com/vector-db-comparison>)

There is probably no best vector database, but there will be one that is suitable for our project. Some criteria that can guide our choice are as follows:

- **Open source or private source:** Open source databases offer transparency and the ability to customize the system. They usually have an active community and no associated costs. Private source databases, on the contrary, can be an expensive solution but often have dedicated support. Similarly, it is important to check the license; it may not be compatible with your product.
- **Language support:** Vector databases are generally compatible with major programming languages (Python, Java, and C), but for our project, we may need a database compatible with another language (Rust, Go, Scala, and so on). Also, not all databases are compatible with all libraries. So, it is good to make sure that the system is compatible with our project.
- **Maturity:** Especially for projects that are production-oriented, it is important that the system is stable, scalable, and reliable. Likewise, the system must be supported, adopted by industry, and maintained frequently.
- **Performance:** This is influenced by two parameters:
 - **Insertion speed:** the rate at which vectors can be added to a database (which affects latency). This especially impacts applications that are in real time or have a large user base. Some databases implement techniques such as batch processing (efficient partitioning of various data packets), parallelization (distribution of tasks across various nodes, especially important for the cloud), or data partitioning (the dataset is divided into segments in order to conduct insertions and deletions at the same time).
 - **Query speed:** this refers to the time it takes to find vectors in response to a query. This directly affects the latency time.

There are optimization techniques such as index structures (structuring indexes to make the search faster), caching systems (data that is accessed frequently is saved separately), or specific algorithms. Then there are performance issues related to the specific products, such as the number of concurrent requests that can be made to the dataset. Regulatory compliance and privacy issues are also key. The database should be able to allow differential access (access authorization) and protect vectors from access by unauthorized users.

- **Component integration:** Our system can have several components besides the LLM and embedder (we will see this in more detail in the next chapter). We need to be sure that the database can be integrated with our encoder (and the library we use for the encoder). Also, not all databases accept other components such as a re-ranker, hybrid search, and so on.
- **Cost:** Cloud solutions can have very high costs, so it is recommended to decide in advance what budget you have. The cost could also be associated with the maintenance and support needed to keep the system operational. For example, vectors are valuable data and the cost of backup can grow very quickly.

For example, there are vector libraries that are static (the index data is immutable); this makes it difficult to add new data. Libraries such as **FAISS (Facebook AI Similarity Search)** are not designed for **create, read, update, and delete (CRUD)** operations, so they are not a good choice for dynamic systems where there are multiple users accessing and conducting operations. In contrast, if our database is immutable and we only grant access, FAISS can be a good solution. There are SQL databases that allow support for vectors (an extension of the classic database). These databases allow for efficient indexing of associated metadata. However, these databases are not scalable and often have limitations for vector size (maximum number of dimensions), and performance is lower. SQL databases are a good choice for internal projects that need to be connected to existing enterprise databases (which will probably already be in SQL) but are not a good choice when scalability and performance are important.

Vector-dedicated databases are usually the best solution, especially for performance. In fact, they usually have implemented dedicated and efficient algorithms for searching and indexing vectors. Several of these algorithms are variations of the **approximate nearest neighbors (ANN)** algorithm. ANN usually allows for a good trade-off between efficiency, storage, and accuracy. Approximate search speeds up the search while trying to maintain accuracy (HNSW (Hierarchical Navigable Small World) sacrifices some accuracy but is much faster than an accurate algorithm such as Flat indexing). These databases are also compatible with major languages and libraries (LlamaIndex, LangChain, and so on).

Once we have our vector database populated with our vectors, we need to evaluate how good our system is. Now, we'll see how we can do that.

Evaluating the output

In information retrieval, we are interested in measuring whether a found document is either relevant or irrelevant. Therefore, the most commonly used valuation metrics are precision and recall. Precision is the fraction

of retrieved documents that are relevant, while recall is the fraction of relevant documents that are successfully retrieved. Consider a query in which R represents all relevant documents and NR represents the irrelevant ones in a corpus of documents D . Rq represents the relevant documents found and Dq is the documents returned by the system. We can define the two metrics as follows:

$$precision = \frac{Rq}{Dq} \quad recall = \frac{Rq}{R}$$

The problem with these two metrics is that they do not return goodness of ranking, only whether we are finding all relevant documents or the percentage of relevant documents in the total. Usually, when we use a retriever, we select a number (k) of documents that we use for context (top- k), so we need a metric that takes ranking into account.

Rank	Label	binary_label	precision	recall
0	1	R	1	0.2
1	2	R	1	0.4
2	3	NR	0.666667	0.4
3	4	R	0.750000	0.6
4	5	R	0.800000	0.8
5	6	NR	0.666667	0.8
6	7	NR	0.571429	0.8
7	8	R	0.625000	1.0
8	9	NR	0.555556	1.0
9	10	NR	0.500000	1.0

Figure 5.19 – Rank-specific precision and recall calculated assuming we have five relevant documents in a corpus

We can use the precision-recall curve for this purpose. Whenever we find a relevant document in the rank, recall increases. Precision, on the other hand, increases with documents but decreases with each irrelevant document. By plotting a graph with a curve, we can see this behavior:

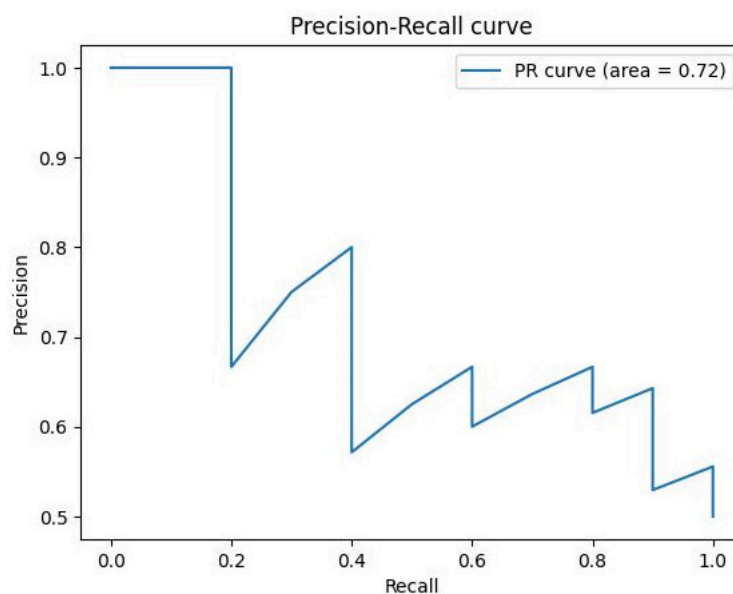


Figure 5.20 – Graph showing the precision and recall curve for the data shown in Figure 5.19

Because the precision goes up and down, we can use an interpolated curve. This curve is less precise but allows us to better understand the be-

havior of the system (and to be able to compare different systems by comparing their curves).

Another metric that is used is **mean average precision (MAP)**. We calculate precision values at the points where a relevant item is retrieved (**average precision** or **AP**) and then average these AP values. Suppose we have retrieved the following list of documents: $[1, 0, 1, 0, 1]$, where 1 means relevant and 0 means not relevant. The precision for each relevant item is as follows:

- The first relevant document is in position 1: Precision (N relevant document retrieved / total of document retrieved) = $1/1 = 1$
- Second relevant document (position 3): Precision = $2/3 \approx 0.67$
- Third relevant item (position 5): Precision = $3/5 = 0.6$

The average precision (AP) value for a single query is:

$$AP = \frac{\sum \text{precision at each relevant item}}{\text{Total number of relevant items}} = \frac{\sum (1 + 0.67 + 0.6)}{3} = 0.76$$

The MAP is the average of all AP values across all the queries. Here, we suppose we have 3 queries – 0.76 , 0.5 , and 0.67 :

$$MAP = \frac{\sum \text{average precision for each query}}{\text{Total number of queries}} = \frac{\sum (0.76 + 0.5 + 0.67)}{3} = 0.64$$

One metric that is specific to question answering is **mean reciprocal rank (MRR)**. MRR is designed to assess the quality of a short-ranked list having the correct answer (usually of human labels). The reciprocal rank is the reciprocal of the rank of the first item relevant to the question. For a set of queries Q , we take the reciprocal ranks and conduct the average:

$$MRR = \frac{1}{Q} \sum_{i=1}^Q \frac{1}{\text{rank}_i}$$

Alternatively, we can evaluate the response after generation as if it were a classification task.

Recently, another way to evaluate RAG pipelines is to use an LLM as a judge of the pipeline. Typically, we must have a dataset that contains ground truth so that LLM evaluates whether the RAG pipeline has found both the necessary steps and the generation is correct. For example, there are different metrics that leverage an LLM as a judge:

- **Faithfulness:** This metric (also called groundedness) measures the factual consistency of the generated answer (range between 0 and 1). An answer is faithful if the claims that are produced in the answer can be inferred from the context. Faithfulness is the ratio of the number of claims in the generated answer that can be inferred from the context to the total number of claims in the generated answer. To find the claims, we need an LLM that evaluates the claims in both the response and the context.
- **Context recall:** This metric refers to how much context is found relative to the ground truth (range of 0 to 1). Ideally, the system should find all the sentences in the ground truth.
- **Context precision:** This metric measures ground-truth relevant items in the context, which are ranked higher (relevant chunks should find themselves higher after retrieval).

- **Context relevancy:** This metric measures the relevance of context to the query. Ideally, our system should only find information that is relevant to the query.
- **Context entities recall:** This metric provides a measure of context recall by specifically analyzing the entities that are found. In other words, it measures the fraction of entities in the ground truth that are found in the context. This metric is useful when we are interested in the system specifically finding entities (for example, the context needs to find medical entities such as diseases, drugs, or other parameters).
- **Answer correctness:** This metric focuses on critically evaluating whether the answer is correct. To have a high value, the system must generate answers that are semantically similar to ground truth but also factually correct.
- **Summarization score:** This metric assesses how well a summary captures the important information that is present in the context. The answer is a kind of summary of the context, and a good summary must contain the important information.
- **Answer relevance:** This metric calculates how relevant the generated response is in response to a prompt. A low score means that the response is incomplete or contains redundant information.
- **Fluency:** This metric assesses the quality of individual sentences generated.
- **Coherence:** This metric assesses whether the entire response is a cohesive corpus (avoids the response being a group of unconnected sentences).

These metrics require that there be an evaluator who is either human or an LLM. They are not simply statistical values but require that the response (and/or the found context) be evaluated critically.

RAG is also often discussed as an alternative to fine-tuning; thus, it is important to compare them, which we'll do next.

Comparison between RAG and fine-tuning

RAG and fine-tuning are often compared and considered techniques in opposition. Both fine-tuning and RAG have a similar purpose, which is to provide the model with knowledge it did not acquire during training. In general, we can say that there are two types of fine-tuning: one directed at adapting a model to a specific domain (such as medicine, finance, or other) and one directed at improving the LLM's ability to perform a particular task or class of tasks (math problem solving, question answering, and so on).

There are several differences between fine-tuning and RAG:

- **Knowledge updates:** RAG allows a direct knowledge update (of both structured and unstructured information). This update can be dynamic for RAG (information can be saved and deleted in real time). In contrast, fine-tuning requires retraining because the update is static (impractical for frequent changes).
- **Data processing:** Data processing is minimal for RAG, while fine-tuning requires quality datasets (datasets with not enough examples will not be able to help with noticeable improvements).

- **Model customization:** RAG provides additional information to the LLM but does not change its behavior or writing style. Fine-tuning allows changes in model behavior, writing style, and even new skills.
- **Interpretability:** RAG increases the interpretability of the system and allows tracking of responses and sources used. Fine-tuning makes the model less interpretable and makes it more difficult to track whether a behavior comes from fine-tuning or the original model.
- **Computational resources:** RAG has an additional cost associated with the encoder and databases (finding information, embedding data, storing information, and so on). This can increase latency cost (you have to add retrieval time to generation time). Fine-tuning requires preparing and curating quality datasets (acquiring certain datasets can be expensive and labor-intensive). In addition, fine-tuning has computational costs associated with model retraining, but it provides lower latency. Moreover, RAG requires less technical expertise than fine-tuning.
- **Reducing hallucinations:** RAG is inherently less prone to hallucinations and allows the tracking of which context is used. Fine-tuning can reduce hallucinations (but, often, fine-tuned LLMs do still exhibit hallucinations).
- **Ethical and privacy issues:** In the case of RAG, we must be careful how the information stored in the database is saved. The database must be protected against potential intrusions and prevent leakage. For fine-tuning, it is important to take care of the training dataset and prevent it from containing sensitive data.

RAG is the best system when we need a dynamic system that can adapt to real-time data or we have large amounts of internal data that are not well structured, though. Likewise, RAG is preferred when it is important to minimize hallucinations, as we need to track the sources of the response when transparency is vital. Fine-tuning is a priority choice when we need to have the model develop specific skills or want to align the model with a particular style of writing or vocabulary.

Some practical examples show where it is best to choose fine-tuning or RAG:

- **Summarization** is important, especially for cases where the domain is highly specialized. It is more critical that the model best understands the context, so fine-tuning is more appropriate.
- **Question answering** is an extremely relevant task that is often used in different domains (questions about documentation, products, and so on). In this case, reducing hallucinations and transparency are critical aspects but customization is much less important. RAG is therefore a better choice.
- **Code generation** is a task that requires that the code base be dynamic; at the same time, it is important to reduce hallucinations and errors. On the other hand, we need the model to be adapted as much as possible to the task. So, both RAG and fine-tuning would be of benefit.

As the last example shows, there are cases where both fine-tuning and RAG would be beneficial. The two systems are not necessarily in opposition to each other. We can conduct fine-tuning of both the LLM and the embedder. So, having a system built with RAG *plus* fine-tuning the LLM (or even the RAG encoder) would be beneficial. Targeted fine-tuning to improve capabilities for specific tasks in the domain of our data can lead

to better performance. This allows for a dynamic information system (where we conduct the RAG update) but adapts the style of the LLM to the domain. The LLM will also be more able to understand and use the context that is provided by RAG. Another case where fine-tuning the model is beneficial is when the found data is in a specific format (code, tables, XML, or other formats) – we can then use an LLM that is adapted to these specific formats (instead of a naive LLM). Additionally, we can optimize our LLM for the task of generating answers by exploiting the provided context. In this case, the LLM can be pushed to make the best use of the found context.

The encoder can also be fine-tuned. Fine-tuning the embedder with a specific dataset increases the contextual understanding of the model (remember that the encoder is a language model that has some contextual understanding of the data). This allows the LLM to better understand the domain-specific nuances of the data, leading to better contextual retrieval (chunks that are more relevant to the query). Of course, it is not always possible to obtain datasets for this task. However, there are approaches in which synthetic data is generated or large LLMs are used to create datasets appropriate for the encoder fine-tuning.

Fine-tuning an embedder is much cheaper than training it from scratch. Today, many models are available, and libraries such as Sentence Transformer facilitate the process of fine-tuning. These models have been pre-trained as embedders, but during fine-tuning, we want to better fit them to our particular data type or domain. Typically, this fine-tuning is conducted with supervised learning using datasets similar to those used for embedder training (with positive and negative examples).

Now that we have seen the main components, in the next section, we will assemble the system.

Using RAG to build a movie recommendation agent

In the previous sections, we discussed what RAG is and how this system can be used to reduce hallucinations or extend model knowledge. As we mentioned, this system is composed of the following components:

- An LLM to generate the answer
- An encoder/retriever that transforms queries and documents into vectors
- A vector database where we save our vectors

We have, in this case, a dataset of movies and their description, and we want to create a system that, by asking a natural language question, will suggest the most suitable movies based on the information we've provided. Our LLM has no specific knowledge of the movies, and its parametric memory does not contain information about the latest releases. Therefore, RAG is a good system to supplement its knowledge.

1. The first step is to obtain a corpus of chunks. Having taken a corpus of documents, we have to reduce it into chunks. A good compromise is to use a text splitter that preserves semantic information (without the

need to use an LLM). In this case, we use a chunker with a size of 1,500 characters. We apply it to the column of our data frame:

```
text_splitter = NLTKTextSplitter(chunk_size=1500)
def split_overview(overview):
    if pd.isna(overview):
        return []
    return text_splitter.split_text(str(overview))
df['chunks'] = df['text_column'].apply(split_overview)
```

2. Next, we need to transform our chunks into vectors. In this case, we are using **all-MiniLM-L6-v2** as an embedder. **all-MiniLM-L6-v2** is a small model that has a good balance between embedding quality and speed. In fact, the model has only 22.7 million parameters, which makes it extremely fast and a good initial choice for testing one's RAG pipeline.

In this case, we do the following:

1. Load the model (the embedder)
2. Create a function to conduct the embedding
3. Conduct the embedding of the various vectors

```
embedder = SentenceTransformer('all-MiniLM-L6-v2')
def encode_chunk(chunk):
    if not isinstance(chunk, str) or chunk.strip() == "":
        return None
    return embedder.encode(chunk).tolist()
chunked_df['embeddings'] = chunked_df['chunks'].apply(encode_chunk)
```

3. At this point, we need to save the vectors we have created to a database. A popular choice as a vector database is Chroma. In this case, we need to do the following:

1. Start the Chroma client
2. Create a new collection
3. Insert the chunks

```
chunked_df.dropna(subset=['embeddings'], inplace=True)
client = chromadb.Client()
collection = client.create_collection(name='movies')
for idx, row in chunked_df.iterrows():
    collection.add(
        ids=[str(idx)],
        embeddings=[row['embeddings']],
        metadatas=[{
            'original_title': row['original_title'],
            'chunk': row['chunks']
        }]
    )
```

IMPORTANT NOTE

Note that we can add metadata (in this case, the title of the film).

We have now implemented only the first part of the pipeline we described earlier. At present, we have a database with the RAG vectors.

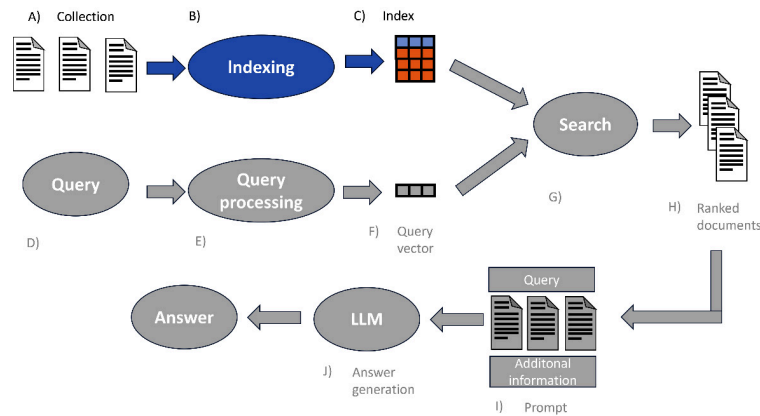


Figure 5.21 – Vector database pipeline

Now, we need to create a pipeline when a query arrives (in inference). In this case, we want to create a vector for the query and search the top k similar vectors in our vector database. This will return the text to an LLM and then generate an answer to our query.

4. We use the same embedder model we used to find the chunks, though, so we need a function that does the following:

1. Creates a vector for the query
2. Finds the most similar documents
3. Returns the associated text

```
def retrieve_documents(query, collection, top_k=5):
    query_embedding = embedder.encode(query).tolist()
    results = collection.query( query_embeddings=[query_embedding], n_results=
    chunks = []
    titles = []
    for document in results['metadatas'][0]:
        chunks.append(document['chunk'])
        titles.append(document['original_title'])
    return chunks, titles
```

At this point, we need to generate the answers, so we need an LLM.

The idea is to provide the LLM with clear instructions, so we create a simple prompt that explains the task to the model. We also provide the model with both the context and the question:

```
tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-Instruct-v0.1")
model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-Instruct-v0.1",
    device_map='auto')
text_generation_pipeline = pipeline(
    model=model,
    tokenizer=tokenizer,
    task="text-generation",
    return_full_text=True,
    max_new_tokens=800)

def generate_answer(query, chunks, titles, text_generation_pipeline):
    context = "\n\n".join([f"Title: {title}\nChunk: {chunk}" for title, chunk in
    prompt = f"""[INST]
    Instruction: You're an expert in movie suggestions. Your task is to analyze c
    {query}
    Here is the context to help you:
    {context}
    [/INST]"""
    generated_text = text_generation_pipeline(prompt)[0]['generated_text']
    return generated_text
```

Now, we can test it. We can ask the system a question and see whether it generates a response:

```
client = chromadb.Client()
collection = client.get_collection(name='movies')
query = "What are some good movies to watch on a rainy day?"
top_k = 5
chunks, titles = retrieve_documents(query, collection, top_k)
print(f"Retrieved Chunks: {chunks}")
print(f"Retrieved Titles: {titles}")
if chunks and titles:
    answer = generate_answer(query, chunks, titles, text_generation_pipeline)
    print(answer)
else:
    print("No relevant documents found to generate an answer.")
```

We now have a complete system. The principle applies to any corpus of documents.

Summary

RAG is one of the fastest-growing paradigms in the field of LLMs. Eliminating hallucinations is one of the most important challenges and one of the most problematic constraints for LLMs and agents to be put into production. RAG is also a flexible system that has several advantages over fine-tuning. As we have seen, this system can be updated frequently with minimal cost and is compatible with different types of data. The naïve RAG is the basic system, consisting of three main components: an LLM, an embedder, and a vector database.

In the next chapter, we will see how this system is evolving. There are now many new additional components, which we will also look at. Despite RAG, sometimes the model still hallucinates as if it ignores the context. This is why sophisticated components have evolved, which we will look at in detail. We will also discuss the subtle interplay between parametric memory and context.

Further reading

- Lewis, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020, <https://arxiv.org/abs/2005.11401>
- ANN-Benchmarks, 2024, <https://ann-benchmarks.com/index.html>
- Hamming Distance between Two Strings: <https://www.geeksforgeeks.org/hamming-distance-two-strings/>

Table of contents collapsed